



SECURITY ASSESSMENT VeSteelV2 Staking



July 23, 2026

Audit Status: Conditional - Remediation Advised

RISK ANALYSIS | VeSteelV2.

■ Classifications of Manual Risk Results

Note: Risk classifications follow CVSS v4.0 standards (<https://www.first.org/cvss/v4.0/specification-document>). Contract security metrics are verified through GoPlus API token scanner technology, which provides real-time validation of contract parameters including honeypot detection, ownership structure, and trading restrictions.

Classification	CVSS Range	Description
 Critical	9.0 - 10.0	Critical vulnerabilities that pose immediate and severe risks requiring urgent attention.
 High	7.0 - 8.9	High-priority issues that could lead to significant security breaches or financial loss.
 Medium	4.0 - 6.9	Medium-severity findings that should be addressed to improve contract security.
 Low	0.1 - 3.9	Low-risk items or best practice suggestions with minimal security impact.
 Informational	0.0	Informational findings about detected functions or contract features.

■ Manual Code Review Risk Results

Contract Security	Description
 Buy Tax	N/A
 Sale Tax	N/A
 Cannot Buy	N/A
 Cannot Sale	N/A
 Max Tax	N/A
 Modify Tax	N/A
 Fee Check	Not Applicable
 Is Honeypot?	Not Applicable
 Trading Cooldown	Not Detected
 Enable Trade?	N/A

Contract Security	Description
● Pause Transfer?	Not Detected
● Max Tx?	Not Detected
● Is Anti Whale?	Not Detected
● Is Anti Bot?	Not Detected
● Is Blacklist?	Not Detected
● Blacklist Check	Not Detected
● Is Whitelist?	Not Detected
● Can Mint?	N/A
● Is Proxy?	Not Detected
● Can Take Ownership?	Dev key controls mine/tiers/seed (see CFG20)
● Hidden Owner?	Not Detected
● Owner	Dev-controlled (devAddr; no Ownable)
● Self Destruct?	Not Detected
● External Call?	Present (trusted wiring)
● Other?	Lock tiers, permanent burn lock (4x), native-currency reward accumulator, dev migration seeding
● Holders	N/A
● Audit Confidence	Elevated Risk - Remediation Advised
● Authority Check	Renounced
● Freeze Check	N/A

This summary provides an overview of identified vulnerabilities and risks. See the full report for detailed methodology and recommendations.



VeSteelV2

Executive Summary

TYPES

DeFi

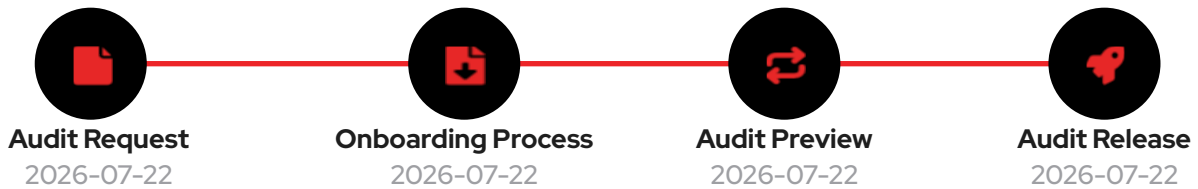
ECOSYSTEM

Robinhood Chain

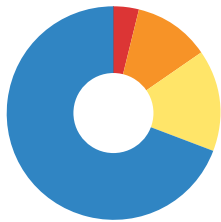
LANGUAGE

Solidity

Timeline



Vulnerability Summary



26

Total Findings

0

Resolved

8

Pending

26

Unresolved

Severity	Resolution Status	Description
0 Critical		Critical risks are the most severe and can have a significant impact on the smart contracts functionality, security, or the entire system. These vulnerabilities can lead to the loss of user funds, unauthorized access, or complete system compromise.
1 High	0 Resolved, 1 Pending	High-risk vulnerabilities have the potential to cause significant harm to the smart contract or the system. While not as severe as critical risks, they can still result in financial losses, data breaches, or denial of service attacks.
3 Medium	0 Resolved, 3 Pending	Medium-risk vulnerabilities pose a moderate level of risk to the smart contracts security and functionality. They may not have an immediate and severe impact but can still lead to potential issues if exploited. These risks should be addressed to ensure the contracts overall security.
4 Low	0 Resolved, 4 Pending	Low-risk vulnerabilities have a minimal impact on the smart contracts security and functionality. They may not pose a significant threat, but it is still advisable to address them to maintain a robust security posture.

Severity

Resolution Status

Description

 18 Informational

18 Resolved, 0 Pending



Informational risks are not actual vulnerabilities but provide useful information about potential improvements or best practices. These findings may include suggestions for code optimizations, documentation enhancements, or other non-critical areas for improvement.

Industry Standards Compliance

This audit follows internationally recognized security standards to provide comprehensive assurance.

PROJECT OVERVIEW | VeSteelV2.

Token Summary

Parameter	Result
Address	0xD5116ca699eD6CA186BC07f46B3c851D1A483aa2
Name	VeSteelV2
Token Tracker	VeSteelV2 (veSTEEL)
Decimals	N/A
Supply	N/A (locking contract)
Platform	Robinhood Chain
Compiler	v0.8.33+commit.64118f21
Contract Name	VeSteelV2
Optimization	Yes with 200 runs
LicenseType	MIT (SPDX)
Language	Solidity
Codebase	https://robinhoodchain.blockscout.com/address/0xD5116ca699eD6CA186BC07f46B3c851D1A483aa2?tab=contract

Main Contract Assessed

Name	Contract	Live
VeSteelV2	0xD5116ca699eD6CA186BC07f46B3c851D1A483aa2	Yes
VeinToken (STEEL)	0x0AF77e27F535256965944E617a386570f5C0432a	Yes
SteelMineV2 (Mine)	0xB089d11432F219495A4278acd6446B7Faefa2bA6	Yes

TestNet Contract Was Not Assessed

Solidity Code Provided

SolID	File Sha-1	FileName
VeSteelV2	41df1cdab6408de5615ec42bddde30b029cd6d1c	VeSteelV2.sol

TECHNICAL FINDINGS

Smart contract security audits classify risks into several categories: Critical, High, Medium, Low, and Informational. These classifications help assess the severity and potential impact of vulnerabilities found in smart contracts.

Classification of Risk

Severity	CVSS Range	Description
 Critical	9.0 - 10.0	Immediate danger. Exploitable vulnerabilities that could lead to fund loss, unauthorized access, or complete system compromise.
 High	7.0 - 8.9	Significant security risk. Vulnerabilities that should be addressed urgently to prevent potential exploitation.
 Medium	4.0 - 6.9	Moderate security risk. Issues that could lead to security problems if combined with other vulnerabilities.
 Low	0.1 - 3.9	Minor security concern. Best practice violations or low-impact issues.
 Informational	0.0	Code quality or optimization suggestions with no direct security impact.

Slither Static Analysis Results

Slither is a Solidity static analysis framework that runs a suite of vulnerability detectors to identify potential security issues, code quality problems, and best practice violations. It provides comprehensive security analysis with confidence ratings for each finding. For full detector documentation, visit: <https://github.com/crytic/slither/wiki/Detector-Documentation>

Impact Level	Count	Description
High	1	Critical security vulnerabilities requiring immediate attention (e.g., reentrancy, unchecked transfers)
Medium	1	Significant issues that should be addressed (e.g., dangerous comparisons, arithmetic issues)
Low	4	Minor issues and best practice violations (e.g., missing events, naming conventions)
Informational	27	Code quality and optimization suggestions (e.g., unused variables, gas optimizations)
TOTAL	33	Total findings across all severity levels

Analysis Summary

Slither 0.11.5 (solc 0.8.33) reported 33 results. Triaged: reentrancy-eth in withdraw() and reentrancy-no-eth in creditDevLock() (external call before state update - guarded by nonReentrant / trusted token, captured as CFG19); low-level native-currency .call in _claim/claimDev; missing-zero-check and immutable-states on devAddr (CFG03); timestamp comparison in withdraw (CFG17). Remaining items are OpenZeppelin pragma/solc-version/assembly/events-maths informational notices. No arbitrary-send-to-attacker, PRNG or uninitialized-storage findings.

Remediation Approach

All High and Medium impact findings from Slither have been manually reviewed and mapped to CFG test cases where applicable. Each finding has been analyzed in the context of the contract's business logic to determine actual exploitability. Detailed analysis of specific findings is available in the Technical Findings section of this audit report.

Deprecation Notice

The Smart Contract Weakness Classification Registry (SWC Registry) was deprecated in September 2023. CFGNinja audits now use Slither for static analysis, which provides more comprehensive and up-to-date vulnerability detection. SWC was an implementation of the weakness classification scheme proposed in EIP-1470, loosely aligned to the Common Weakness Enumeration (CWE). For historical reference: <https://swcregistry.io/>

I Detailed Security Findings

Each finding lists its severity, CVSS, location, description, the affected source excerpt (verbatim from the verified Blockscout source), our recommendation, and a suggested fix where the remediation is code-expressible.

CFG03 | Missing Zero-Address Validation.

LOW · CVSS 2.5 · Validation

Location: VeSteelV2.sol : constructor (L67-L70)

Description

Neither the constructor's `_devAddr` nor `setMine`'s `_mine` is checked against the zero address (Slither: missing-zero-check; `devAddr` could also be immutable). A zero `devAddr` disables dev-gated functions; a zero mine set during the open window is a griefing outcome.

Affected Code

```
constructor(ISteelBurnable _steel, address _devAddr) {  
    steel = _steel;  
    devAddr = _devAddr;           // no zero-address check; not immutable  
}
```

Recommendation

Add `require(addr != address(0))` guards and mark `devAddr` immutable.

Suggested Fix

```
address public immutable devAddr;  
constructor(ISteelBurnable _steel, address _devAddr) {  
    require(_devAddr != address(0), "zero");  
    steel = _steel; devAddr = _devAddr;  
}
```

CFG05 | Missing Events on Admin Changes.

LOW · CVSS 2.5 · Logging

Location: VeSteelV2.sol : setMine / setTiers (L72-L82)

Description

setTiers and setMine change trust-critical configuration but emit no events, so tier retunes and mine re-pointing are invisible to off-chain monitors and explorer event feeds.

Affected Code

```
function setMine(address _mine) external {
    require(msg.sender == devAddr || mine == address(0), "only dev");
    mine = _mine;                // no event emitted
}
```

Recommendation

Emit TiersUpdated and MineSet events.

Suggested Fix

```
event MineSet(address indexed mine);
event TiersUpdated(uint256[5] durs, uint256[5] mults);
// emit MineSet(_mine); / emit TiersUpdated(durs, mults);
```

CFG12 | No Timelock on Dev Parameters.

MEDIUM · CVSS 5.0 · Centralization

Location: VeSteelV2.sol : setTiers (L78-L82)

Description

setTiers(durations, multipliers) lets the dev retune all lock durations and multipliers at any time with immediate effect and no timelock. Existing positions keep the weight computed at lock time, so retuning only affects future locks, but a user deciding to lock has no guarantee the advertised terms hold next block. Combined with the ability to repoint the mine, this is a meaningful centralization surface.

Affected Code

```
function setTiers(uint256[5] calldata durs, uint256[5] calldata mults) external {
    require(msg.sender == devAddr, "only dev");
    TIER_DURATION = durs;           // immediate effect, no timelock, no event
    TIER_MULT = mults;
}
```

Recommendation

Add a timelock (e.g. 48h) to setTiers and any mine re-pointing, and emit an event on every change.

Suggested Fix

```
function setTiers(uint256[5] calldata durs, uint256[5] calldata mults) external {
    require(msg.sender == timelock, "only timelock"); // 48h delay
    TIER_DURATION = durs; TIER_MULT = mults;
    emit TiersUpdated(durs, mults);
}
```

CFG16 | Dev-Controlled Migration Seeding.

MEDIUM · CVSS 5.8 · Centralization

Location: VeSteelV2.sol : seedBurnLock (L145-L148)

Description

Before finalizeSeed() is called, seedBurnLock(owner, weight) and seedDevWeight(weight) let devAddr mint arbitrary reward weight to any address - including the dev's own permanent position - without depositing or burning any STEEL. Until the seed window is closed the dev can capture an arbitrary share of the reward stream. This is a migration convenience but concentrates significant trust in a single unlocked key.

Affected Code

```
function seedBurnLock(address owner, uint256 weight) external returns (uint256 id) {
    require(msg.sender == devAddr && !seeded, "seed closed");
    id = _open(owner, 0, uint128(weight), 0, true); // arbitrary weight, no deposit
}
```

Recommendation

Call finalizeSeed() immediately after migration and publish the seeded weights. Hold devAddr in a multisig and emit per-address seed events for on-chain verification.

Suggested Fix

```
// close the migration window immediately after seeding, and log seeds:
function finalizeSeed() external {
    require(msg.sender == devAddr, "only dev");
    seeded = true;
    emit SeedFinalized();
}
```

CFG17 | Block Timestamp Lock Enforcement.

LOW · CVSS 2.0 · Security

Location: VeSteelV2.sol : withdraw (L187-L188)

Description

Lock expiry is enforced with block.timestamp. Validators can nudge the timestamp by a few seconds; for the 1h-6mo tiers this is negligible (well under 0.01% of the shortest lock).

Affected Code

```
if (l.burned || l.end == 0) revert Permanent();  
if (block.timestamp < l.end) revert StillLocked(); // validator +/- secs
```

Recommendation

No action required unless sub-minute locks are introduced.

Suggested Fix

```
// Acceptable at 1h-6mo tiers (skew << lock). No change required unless  
// sub-minute locks are introduced.
```


CFG19 | Checks-Effects-Interactions Ordering.

LOW · CVSS 3.5 · Security

Location: VeSteelV2.sol : withdraw (L184-L195)

Description

withdraw() calls _claim() (which performs a low-level native-currency .call to the lock owner) BEFORE decrementing totalWeight and deleting the lock, and creditDevLock() performs an external steel.burn() before updating devWeight/devRewardDebt (Slither: reentrancy-eth / reentrancy-no-eth). The user entrypoints carry nonReentrant and steel is the trusted immutable token, so these are not currently exploitable, but the ordering violates checks-effects-interactions and creditDevLock itself has no nonReentrant modifier.

Affected Code

```
_claim(l, lockId);                // interaction (native-currency call) FIRST
uint256 amount = l.amount;
totalWeight -= l.weight;          // effects AFTER the external call
delete locks[lockId];
IERC20(address(steel)).safeTransfer(msg.sender, amount);
```

Recommendation

Reorder so all state is updated before any external call, and add nonReentrant to creditDevLock for defense in depth.

Suggested Fix

```
uint256 amount = l.amount;        // effects first
totalWeight -= l.weight;
delete locks[lockId];
_claim(l, lockId);                // interaction last (CEI)
IERC20(address(steel)).safeTransfer(msg.sender, amount);
```

CFG20 | Unprotected Mine Initialization - Anyone Can Set the Mine.

HIGH · CVSS 7.8 · Centralization / Access Control

Location: VeSteelV2.sol : setMine (L72-L75)

Description

setMine(address) is guarded by 'require(msg.sender == devAddr || mine == address(0))'. Because of the '|| mine == address(0)' clause, while mine is still unset ANY external account can register an arbitrary address as the trusted mine. The mine can call creditDevLock, which burns STEEL held by this contract and inflates devWeight/totalWeight, diluting every locker's pro-rata share. An attacker who front-runs the legitimate setMine (or acts before the deployer sets it) can grief the reward stream and destroy contract-held STEEL. The devAddr clause also allows the mine to be repointed later at any time with no timelock. Whether mine is already set on the live deployment could not be confirmed on-chain.

Affected Code

```
function setMine(address _mine) external {
    require(msg.sender == devAddr || mine == address(0), "only dev");
    mine = _mine;                // anyone can set while mine == 0
}
```

Recommendation

Remove the '|| mine == address(0)' escape and gate setMine strictly on devAddr (or a one-time initializer that reverts once set). Set the mine atomically in the deployment script before the contract is otherwise usable. Add a timelock on any later re-pointing.

Suggested Fix

```
function setMine(address _mine) external {
    require(msg.sender == devAddr, "only dev");
    require(_mine != address(0), "zero");
    mine = _mine;
    emit MineSet(_mine);
}
```

CFG23 | Anonymous Team - No KYC Verification.

MEDIUM · CVSS 5.0 · Team Transparency

Location: Project team

Description

No team identities or KYC are disclosed. For a contract that custodies locked STEEL and streams native currency under a single unlocked dev key, an anonymous team is a heightened trust concern.

Recommendation

Complete KYC with a recognized provider and disclose at least a team-lead identity.

FINDINGS

In this document, we present the findings and results of the smart contract security audit. The identified vulnerabilities, weaknesses, and potential risks are outlined, along with recommendations for mitigating these issues. It is crucial for the team to address these findings promptly to enhance the security and trustworthiness of the smart contract code.

Severity	Found	Pending	Resolved
 Critical	0	0	0
 High	1	1	0
 Medium	3	3	0
 Low	4	4	0
 Informational	18	0	18
Total	26	8	0

In a smart contract, a technical finding summary refers to a compilation of identified issues or vulnerabilities discovered during a security audit. These findings can range from coding errors and logical flaws to potential security risks. It is crucial for the project owner to thoroughly review each identified item and take necessary actions to resolve them. By carefully examining the technical finding summary, the project owner can gain insights into the weaknesses or potential threats present in the smart contract. They should prioritize addressing these issues promptly to mitigate any risks associated with the contract's security. Neglecting to address any identified item in the security audit can expose the smart contract to significant risks. Unresolved vulnerabilities can be exploited by malicious actors, potentially leading to financial losses, data breaches, or other detrimental consequences. To ensure the integrity and security of the smart contract, the project owner should engage in a comprehensive review process. This involves understanding the nature and severity of each identified item, consulting with experts if needed, and implementing appropriate fixes or enhancements. Regularly updating and maintaining the smart contract's codebase is also essential to address any emerging security concerns. By diligently reviewing and resolving all identified items in the technical finding summary, the project owner can significantly reduce the risks associated with the smart contract and enhance its overall security posture.

SOCIAL MEDIA CHECKS | VeSteelV2.

Social Media	URL	Result
Website	https://www.steelrh.fun/	Pass
Telegram	https://discord.com/invite/DUVvVKSyR	Pass
Twitter	https://x.com/steeldotfun	Pass
Facebook		N/A
Reddit	N/A	N/A
Instagram	N/A	N/A
CoinGecko		N/A
Github	N/A	N/A
CMC		N/A
Email		Contact
Other	https://discord.com/invite/DUVvVKSyR	Pass

From a security assessment standpoint, inspecting a project's social media presence is essential. It enables the evaluation of the project's reputation, credibility, and trustworthiness within the community. By analyzing the content shared, engagement levels, and the response to any security-related incidents, one can assess the project's commitment to security practices and its ability to handle potential threats.

Social Media Information Notes:

Auditor Notes: Project website (steelrh.fun), X/Twitter (@steeldotfun) and Discord confirmed. No CoinGecko/CMC listing at time of audit. Team is anonymous.

Project Owner Notes: Social channels confirmed active.

Assessment Results

Final Audit Score veSTEEL.

Review	Score
Security Score	67
Auditor Score	67

Our security assessment or audit score system for the smart contract and project follows a comprehensive evaluation process to ensure the highest level of security. The system assigns a score based on various security parameters and benchmarks, with a passing score set at 80 out of a total attainable score of 100. The assessment process includes a thorough review of the smart contracts codebase, architecture, and design principles. It examines potential vulnerabilities, such as code bugs, logical flaws, and potential attack vectors. The evaluation also considers the adherence to best practices and industry standards for secure coding. Additionally, the system assesses the projects overall security measures, including infrastructure security, data protection, and access controls. It evaluates the implementation of encryption, authentication mechanisms, and secure communication protocols. To achieve a passing score, the smart contract and project must attain a minimum of 80 points out of the total attainable score of 100. This ensures that the system has undergone a rigorous security assessment and meets the required standards for secure operation.



**AUDIT
FAILED**

Important Notes for veSTEEL

VeSteelV2 (veSTEEL Locking) - Smart Contract Security Audit Report

PROJECT OVERVIEW

VeSteelV2 is the staking / locking contract of the STEEL (steel.fun) protocol on Robinhood Chain. Users lock STEEL for a fixed tier (1 hour, 1 day, 1 week, 1 month, 6 months - STEEL returned on unlock) or permanently BURN STEEL for the maximum multiplier (never withdrawable). Locked positions accrue a pro-rata share, by weight, of real ETH protocol fees streamed from the SteelMineV2 mine. The mine's own dev allocation is credited into the contract as a permanent (burned) position. This report covers the VeSteelV2 contract only.

CONTRACT DETAILS

Name: VeSteelV2 (veSTEEL)

Contract Address: 0xD5116ca699eD6CA186BC07f46B3c851D1A483aa2

Chain: Robinhood Chain (EVM, Cancun)

Compiler: Solidity v0.8.33+commit.64118f21 | Optimization: Enabled, 200 runs | EVM: Cancun

Verification: Verified on Blockscout

Base Libraries: OpenZeppelin SafeERC20, ReentrancyGuard (IERC20)

Reward Asset: Native ETH (streamed from the mine via notifyReward / receive)

Locked Asset: STEEL (VeinToken, 0x0AF77e27...)

Explorer: <https://robinhoodchain.blockscout.com/address/0xD5116ca699eD6CA186BC07f46B3c851D1A483aa2?tab=contract>

LOCKING MECHANICS

- Tiers (duration / multiplier): 1h / 1.0x, 1d / 1.1x, 1w / 1.2x, 1mo / 1.4x, 6mo / 3.0x. Permanent BURN lock = 4.0x (STEEL destroyed).
- Weight = amount * tier multiplier / 1e18. Rewards are distributed with a classic accumulator (accEthPerWeight) and per-lock rewardDebt; $\text{pending} = \text{weight} * \text{accEthPerWeight} / \text{WAD} - \text{rewardDebt}$.
- lock(amount, tier): pulls STEEL via SafeERC20, returns it on unlock after the lock end.
- lockBurn(amount): burns STEEL (via burnFrom) for a single, top-up-able permanent position at 4x.
- withdraw(lockId): after the lock end, claims accrued ETH then returns principal STEEL.
- notifyReward() / receive(): ETH sent in is split pro-rata by weight; if totalWeight is 0 the ETH is parked in `residual` and distributed on the next reward.
- creditDevLock(amount): mine-only; mints STEEL into the contract, burns it, and grows the dev's permanent weight.
- Migration seed functions (seedBurnLock, seedDevWeight, finalizeSeed) let the deployer carry positions over from a previous contract, then close the window forever.

SECURITY CONCERNS

Unprotected Mine Initialization - Anyone Can Set the Mine (HIGH - CFG20)

`setMine(address)` is guarded by `require(msg.sender == devAddr || mine == address(0), "only dev")`. Because of the `|| mine == address(0)` clause, while `mine` is still unset ANY external account can call `setMine` and register an arbitrary address as the trusted mine. The mine is a privileged role: it can call `creditDevLock`, which burns STEEL held by this contract and inflates `devWeight` / `totalWeight`, diluting every locker's pro-rata ETH share. An attacker who front-runs the legitimate `setMine` call (or acts before the deployer sets it) can therefore grief the reward stream and destroy contract-held STEEL. We could not confirm on-chain whether `mine` has already been set on the live deployment; if it has, the window is closed, but the pattern is unsafe by design. Separately, the `devAddr` clause lets the dev re-point the mine at any later time with no timelock.

Recommendation: Remove the `|| mine == address(0)` escape and gate `setMine` strictly on `devAddr` (or a one-time initializer that reverts once set). Set the mine atomically in the deployment script before the contract is otherwise usable. Consider a timelock on any later re-pointing.

Dev-Controlled Migration Seeding (MEDIUM - CFG16)

Before `finalizeSeed()` is called, `seedBurnLock(owner, weight)` and `seedDevWeight(weight)` let

`devAddr` mint arbitrary reward weight to any address - including the dev's own permanent position - without depositing or burning any STEEL (the weight is created out of thin air, "already burned on the old contract"). Until the seed window is closed, the dev can therefore capture an arbitrary share of the ETH reward stream. This is a migration convenience but concentrates significant trust in a single unlocked key.

Recommendation: Call `finalizeSeed()` immediately after migration and publish the seeded weights. Hold `devAddr` in a multisig. Emit events with per-address seeded weight so the community can verify the migration on-chain.

No Timelock on Dev Parameters (MEDIUM - CFG12)

`setTiers(durations, multipliers)` (dev-only) lets the dev retune all lock durations and multipliers at any time with immediate effect and no timelock. Existing positions keep the weight computed at lock time, so retuning only affects future locks - but a user deciding to lock has no guarantee the advertised tier terms will still hold in the next block. Combined with the ability to repoint the mine, this is a meaningful centralization surface.

Recommendation: Add a timelock (e.g. 48h) to `setTiers` and any mine re-pointing, and emit an event on every change.

Checks-Effects-Interactions Ordering (LOW - CFG19)

`withdraw()` calls `\_claim()` (which performs a low-level ETH `.call` to the lock owner) BEFORE it decrements `totalWeight` and deletes the lock, and `creditDevLock()` performs an external `steel.burn()` before updating `devWeight` / `devRewardDebt`. Slither flags these as reentrancy-eth / reentrancy-no-eth. In practice `withdraw`, `claim`, `claimDev` and the user entrypoints carry `nonReentrant`, and `steel` is the trusted immutable token, so these are not currently exploitable - but the ordering violates checks-effects-interactions. Note `creditDevLock` itself has no `nonReentrant` modifier (it relies on the trusted mine/token).

Recommendation: Reorder so all state (totalWeight, lock deletion, rewardDebt, devWeight) is updated before any external call, and add `nonReentrant` to `creditDevLock` for defense in depth.

Missing Events on Admin Changes (LOW - CFG05)

`setTiers` and `setMine` change trust-critical configuration but emit no events, so tier retunes and mine re-pointing are not visible to off-chain monitors or block-explorer event feeds.

Recommendation: Emit `TiersUpdated` and `MineSet` events.

Missing Zero-Address Validation (LOW - CFG03)

Neither the constructor's `\_devAddr` nor `setMine`'s `\_mine` is checked against the zero address (Slither: missing-zero-check; `devAddr` could also be `immutable`). A zero `devAddr` would disable dev-gated functions; a zero `mine` set during the open window would be a griefing outcome.

Recommendation: Add `require(addr != address(0))` guards and mark `devAddr` immutable.

Block Timestamp Lock Enforcement (LOW - CFG17)

Lock expiry is enforced with `block.timestamp` (`block.timestamp < l.end`). Validators can nudge the timestamp by a few seconds; for the 1h-6mo tiers this is negligible (well under 0.01% of the shortest lock).

Recommendation: No action required unless sub-minute locks are ever introduced.

POSITIVE FINDINGS

ReentrancyGuard: Every user-facing fund-movement entrypoint (lock, lockBurn, withdraw, claim, claimDev) is nonReentrant.

SafeERC20: STEEL transfers use SafeERC20, tolerating non-standard ERC20 return behavior.

Sound Reward Accounting: The accEthPerWeight / rewardDebt accumulator is a well-understood pattern; `\_claim` correctly writes rewardDebt before sending ETH, and pending owed ETH is preserved across weight changes in lockBurn / creditDevLock.

Residual Handling: ETH received while totalWeight == 0 is parked in `residual` and folded into the next distribution rather than being lost.

Per-Lock Isolation: A locker whose address reverts on ETH receipt only blocks their own claim - it cannot stall other lockers.

Solidity 0.8.33: Native overflow/underflow protection.

Static Analysis: Slither 0.11.5 (solc 0.8.33) produced 33 raw results; after triage the only non-informational items are the CEI/reentrancy-ordering notes (guarded), the low-level ETH calls, missing-zero-check, and the timestamp usage - all captured above. No

arbitrary-send-to-attacker, no PRNG, no uninitialized-storage findings.
Contract Verified: Source published and verified on Blockscout.

AUDIT METHODOLOGY

Manual line-by-line review of the verified Solidity source (main contract plus all 10 OpenZeppelin additional sources, downloaded from the Robinhood Chain Blockscout API), CFG01-CFG26 framework mapping, and automated static analysis with Slither 0.11.5 compiled against solc 0.8.33 (matching the on-chain compiler) via solc-select with OpenZeppelin remappings. Aderyn was not run (Rust/cargo toolchain not installed on this host); Slither provided full static-analysis coverage.

On-chain state (whether `mine` is already set, whether `finalizeSeed()` has been called, current totalWeight / accEthPerWeight, TVL) could not be independently confirmed because no archive RPC / indexer for Robinhood Chain was available during this review. Findings are based on the verified source code; on-chain-dependent statements are explicitly marked unconfirmed.

Audit Date: July 22, 2026 | Chain: Robinhood Chain (EVM) | Auditor: CFG.NINJA

APPENDIX A: AFFECTED CODE AND SUGGESTED FIXES

The following excerpts are taken verbatim from the verified Blockscout source and are also rendered as monospace boxes in the Detailed Security Findings section of the PDF.

CFG03 - Missing Zero-Address Validation (Low)

Location: VeSteelV2.sol : constructor (L67-L70)

Affected Code:

```
constructor(ISTeelBurnable _steel, address _devAddr) {
    steel = _steel;
    devAddr = _devAddr;          // no zero-address check; not immutable
}
```

Suggested Fix:

```
address public immutable devAddr;
constructor(ISTeelBurnable _steel, address _devAddr) {
    require(_devAddr != address(0), "zero");
    steel = _steel; devAddr = _devAddr;
}
```

CFG05 - Missing Events on Admin Changes (Low)

Location: VeSteelV2.sol : setMine / setTiers (L72-L82)

Affected Code:

```
function setMine(address _mine) external {
    require(msg.sender == devAddr || mine == address(0), "only dev");
    mine = _mine;          // no event emitted
}
```

Suggested Fix:

```
event MineSet(address indexed mine);
event TiersUpdated(uint256[5] durs, uint256[5] mults);
// emit MineSet(_mine); / emit TiersUpdated(durs, mults);
```

CFG12 - No Timelock on Dev Parameters (Medium)

Location: VeSteelV2.sol : setTiers (L78-L82)

Affected Code:

```
function setTiers(uint256[5] calldata durs, uint256[5] calldata mults) external {
    require(msg.sender == devAddr, "only dev");
    TIER_DURATION = durs;          // immediate effect, no timelock, no event
    TIER_MULT = mults;
}
```

Suggested Fix:

```
function setTiers(uint256[5] calldata durs, uint256[5] calldata mults) external {
    require(msg.sender == timelock, "only timelock"); // 48h delay
    TIER_DURATION = durs; TIER_MULT = mults;
    emit TiersUpdated(durs, mults);
}
```

CFG16 - Dev-Controlled Migration Seeding (Medium)

Location: VeSteelV2.sol : seedBurnLock (L145-L148)

Affected Code:

```
function seedBurnLock(address owner, uint256 weight) external returns (uint256 id) {
    require(msg.sender == devAddr && !seeded, "seed closed");
    id = _open(owner, 0, uint128(weight), 0, true); // arbitrary weight, no deposit
}
```

Suggested Fix:**// close the migration window immediately after seeding, and log seeds:**

```
function finalizeSeed() external {
    require(msg.sender == devAddr, "only dev");
    seeded = true;
    emit SeedFinalized();
}
```

CFG17 - Block Timestamp Lock Enforcement (Low)

Location: VeSteelV2.sol : withdraw (L187-L188)

Affected Code:

```
if (l.burned || l.end == 0) revert Permanent();
if (block.timestamp < l.end) revert StillLocked(); // validator +/- secs
```

Suggested Fix:

```
// Acceptable at 1h-6mo tiers (skew << lock). No change required unless
// sub-minute locks are introduced.
```

CFG19 - Checks-Effects-Interactions Ordering (Low)

Location: VeSteelV2.sol : withdraw (L184-L195)

Affected Code:

```
_claim(l, lockId); // interaction (native-currency call) FIRST
uint256 amount = l.amount;
totalWeight -= l.weight; // effects AFTER the external call
delete locks[lockId];
IERC20(address(steel)).safeTransfer(msg.sender, amount);
```

Suggested Fix:

```
uint256 amount = l.amount; // effects first
totalWeight -= l.weight;
delete locks[lockId];
_claim(l, lockId); // interaction last (CEI)
IERC20(address(steel)).safeTransfer(msg.sender, amount);
```

CFG20 - Unprotected Mine Initialization - Anyone Can Set the Mine (High)

Location: VeSteelV2.sol : setMine (L72-L75)

Affected Code:

```
function setMine(address _mine) external {
    require(msg.sender == devAddr || mine == address(0), "only dev");
    mine = _mine; // anyone can set while mine == 0
}
```

Suggested Fix:

```
function setMine(address _mine) external {
    require(msg.sender == devAddr, "only dev");
    require(_mine != address(0), "zero");
    mine = _mine;
    emit MineSet(_mine);
}
```

DISCLAIMER

This audit does not guarantee the absence of all vulnerabilities. This report covers the VeSteelV2 contract only; the VeinToken (STEEL) token and the SteelMineV2 mine are covered in separate reports. The reward stream depends on the SteelMineV2 mine and on ETH being delivered to this contract - those external dependencies are outside the scope of this contract-level review. Users should conduct their own research before interacting with any smart contract.

I Appendix

Finding Categories

Centralization / Privilege

Centralization / Privilege findings refer to either feature logic or implementation of components that act against the nature of decentralization, such as explicit ownership or specialized access roles in combination with a mechanism to relocate funds.

Gas Optimization

Gas Optimization findings do not affect the functionality of the code but generate different, more optimal EVM opcodes resulting in a reduction in the total gas cost of a transaction.

Logical Issue

Logical Issue findings detail a fault in the logic of the linked code, such as an incorrect notion of how `block.timestamp` works.

Control Flow

Control Flow findings concern the access control imposed on functions, such as owner-only functions being invocable by anyone under certain circumstances.

Volatile Code

Volatile Code findings refer to segments of code that behave unexpectedly in certain edge cases that may result in a vulnerability.

Coding Style

Coding Style findings usually do not affect the generated byte-code but rather comment on how to make the codebase more legible and, as a result, easily maintainable.

Inconsistency

Inconsistency findings refer to functions that should seemingly behave similarly yet contain different code, such as a constructor assignment imposing different require statements on the input variables than a setter function.

Coding Best Practices

ERC 20 Coding Standards are a set of rules that each developer should follow to ensure the code meets a set of criteria and is readable by all developers.

Disclaimer

Scope. This report documents the results of a security assessment and smart contract audit performed by Bladepool / CFG NINJA (the "Auditor") for the project specified in this report. The assessment covers only the deliverables and code expressly listed in the engagement; any changes, integrations, or subsequent deployments are outside the scope.

No Warranty. The Auditor performs the assessment using industry-standard practices but makes no representations or warranties, express or implied, including any warranty of merchantability, fitness for a particular purpose, or non-infringement. The Auditor does not guarantee that the audited code is free of bugs, vulnerabilities, or exploitable issues.

Limitation of Liability. To the maximum extent permitted by law, the Auditor and its affiliates, officers, employees, agents, and contractors shall not be liable for any indirect, incidental, special, consequential, or punitive damages, or for any loss of profits, revenue, data, or business opportunities, arising out of or related to the assessment, even if advised of the possibility of such damages. The Auditor's aggregate liability for direct damages is limited to the fees paid for the audit engagement.

Client Responsibility. The project owner/client retains sole responsibility for all decisions and actions taken in connection with the audited code, including fixing identified issues, deploying code to any environment, and applying security mitigations. The Auditor's recommendations are advisory; implementation and verification of fixes are the client's responsibility.

Third-Party Tools and Services. The Auditor may use third-party tools, services, or automated scanners as part of the assessment. Use of such tools is without warranty; the Auditor is not responsible for defects, failures, or inaccuracies arising from third-party services.

Confidentiality and Data Security. The Auditor will treat non-public client information as confidential. However, the Auditor cannot guarantee absolute security for data transmitted over the internet or third-party platforms. Client should take reasonable precautions to protect sensitive information.

Not Financial, Legal, or Investment Advice. The assessment is a technical security review only and is not financial, legal, tax, or investment advice. Clients should consult appropriate professionals for those matters.

Ownership and Governance Notes. If ownership is transferred, held by a multisig, or controlled by a governance contract, the client should document and disclose those arrangements; the Auditor's report reflects the ownership state observed during the engagement but may not reflect subsequent changes.

Governing Law and Counsel. This engagement and any disputes arising from it shall be governed by the laws agreed in the engagement terms. Clients are advised to seek legal counsel before relying on or publishing the report.

Acceptance. By proceeding with this engagement or using this report, the client acknowledges and accepts these terms.

